Lab 1: Mininet, OpenFlow

Anais Sadoudi
M2 CNS - SR - SDN
Université d'Évry, Paris-Saclay

October 2026

Contents

1 Introduction **2**

2 Part A: Setting up the VM **2**
2.1 Hypervisor installation . . . 2
2.2 Mininet VM download and import . . . 2
2.3 First boot . . . 3
2.4 Remote access with SSH . . . 3
2.5 Installation of a newer version of Wireshark . . . 4
2.6 Running Wireshark from the command line . . . 5
2.7 Desktop installation . . . 6

3 Part B: Experimenting with Mininet **6**
3.1 Part B.1: Interact with Hosts and Switches . . . 8
3.2 Part B.2: Changing Topology Size and Type . . . 18
3.3 Part B.3: Custom Topologies . . . 23
3.4 Part B.4: ID = MAC . . . 26
3.5 Part B.5: Links Variation . . . 30
3.6 Part B.6: Adjustable Verbosity . . . 33
3.7 Part B.7: Python Interpreter . . . 37
3.8 Part B.8: Link Up/Down . . . 39

4 Conclusion **41**

1 Introduction

Software Defined Networking (SDN) divides the network into a data plane that does the actual packet forwarding and the control plane that makes the decisions on how to forward the data. With SDN, switches are just dumb forwarders that can be programmed to behave a certain way with a standard protocol by a logically centralised controller. OpenFlow [1, 3].

2 Part A: Setting up the VM

2.1 Hypervisor installation

The host machine runs Ubuntu 26.04 LTS. VirtualBox 7.2.6 was installed from the official repositories:

```
sudo apt install -y virtualbox
```

2.2 Mininet VM download and import

The recommended Mininet VM image (Mininet 2.3.0, Ubuntu 20.04.1) was downloaded from the Mininet GitHub releases [5]

The `.ovf` file was imported in VirtualBox (*File* → *Import Appliance*). The VM is configured with 2048 MB of RAM, the maximum video memory.

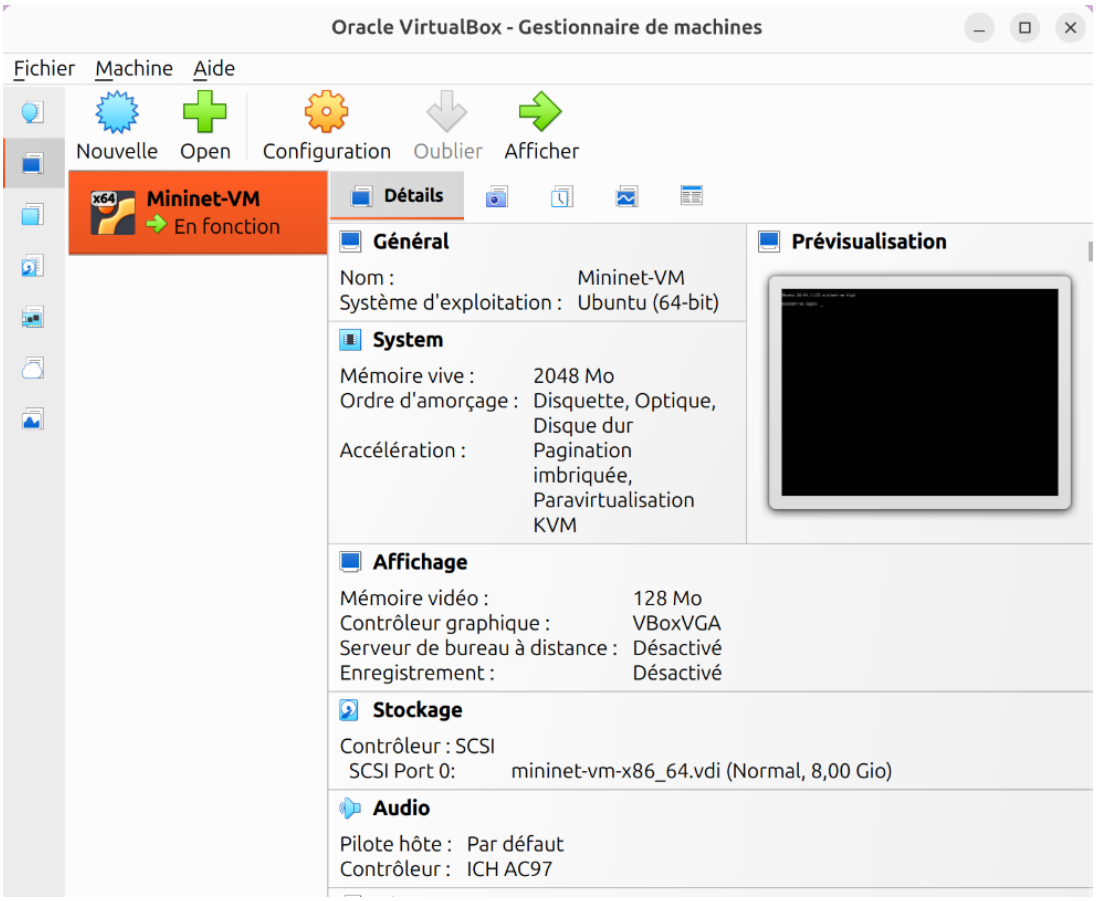


Figure 1: Mininet VM imported and configured in VirtualBox.

2.3 First boot

The VM boots on Ubuntu 20.04.1. After logging in with the `mininet`/`mininet` credentials, the installed version was checked with `mn -version` (Mininet 2.3.0). The command `ip a` shows the loopback interface and `eth0` with the address 10.0.2.15/24, given by the VirtualBox NAT adapter, which provides Internet access to the VM.

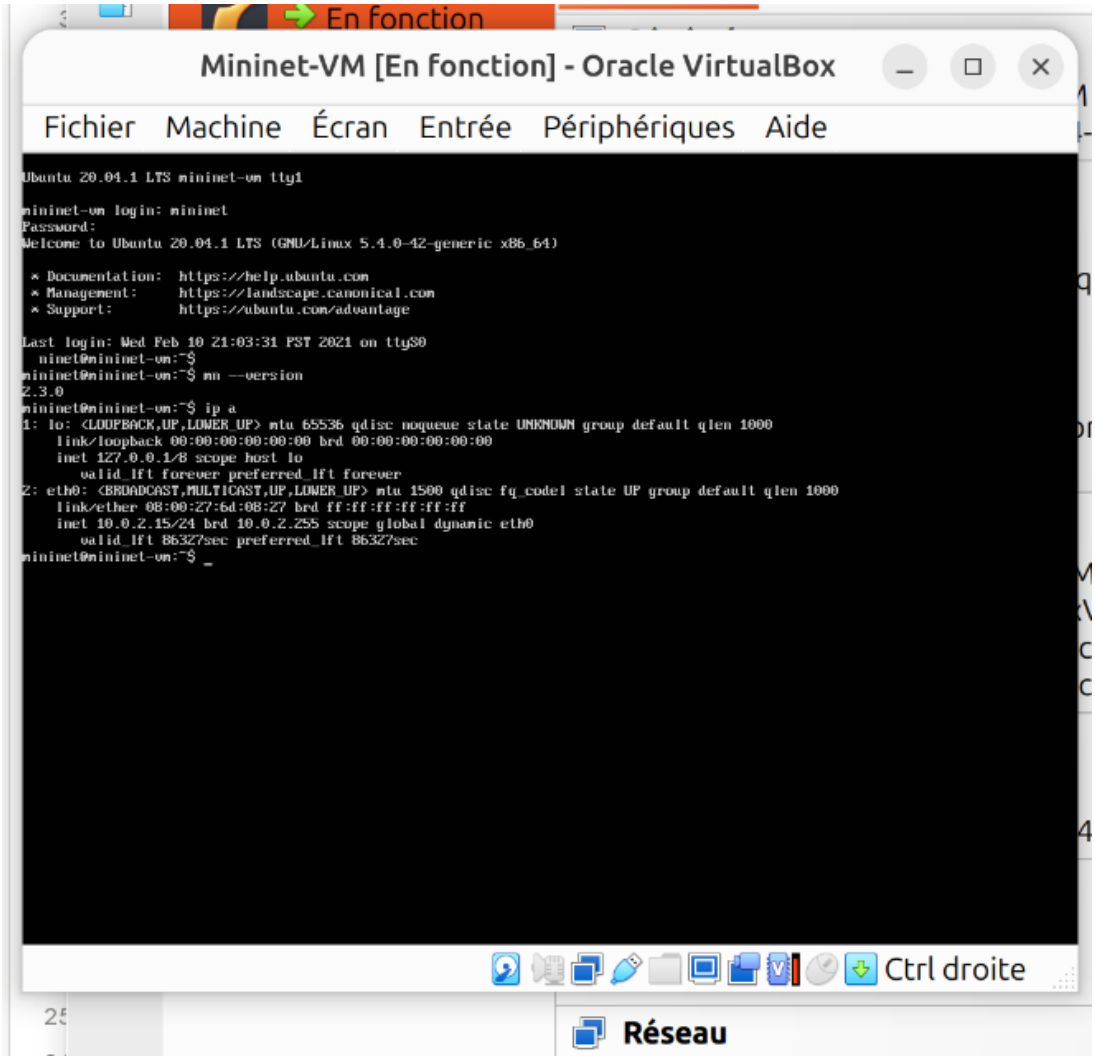


Figure 2: First login in the Mininet VM, mn –version and ip a.

2.4 Remote access with SSH

The lab statement suggests connecting to the VM with `ssh -X [user]@[Guest IP]`. With the default VirtualBox NAT adapter, the guest address (10.0.2.15) is not reachable from the host. A NAT port forwarding rule was therefore added in VirtualBox (*Settings* → *Network* → *Adapter 1* → *Port Forwarding*): TCP port 2222 of the host (127.0.0.1) is forwarded to port 22 (SSH) of the guest. The VM is then reached from a terminal of the host with:

```
ssh -X -p 2222 mininet@127.0.0.1
```

The `-X` option enables X11 forwarding: graphical applications started in the VM (such as `xterm` or Wireshark) are displayed on the host desktop.

2.5 Installation of a newer version of Wireshark

The commands of the lab statement were executed in the VM. Several issues were encountered and solved:

- `python-software-properties` could not be installed: this Python 2 package is obsolete on Ubuntu 20.04 and is replaced by `software-properties-common`, which was installed instead.
- The installation first failed with `404 Not Found` errors, because the package lists of the VM image dated from 2021 and referred to package versions no longer available on the Ubuntu mirrors. Running `sudo apt-get update` first solved the problem.
- `wireshark-gtk` could not be installed : the GTK interface was dropped in Wireshark 3.0 and this package is only a transitional package.

```
sudo apt-get remove wireshark
sudo apt-get update
sudo apt-get install software-properties-common
sudo add-apt-repository ppa:wireshark-dev/stable
sudo apt-get update && sudo apt-get install wireshark
wireshark --version
```

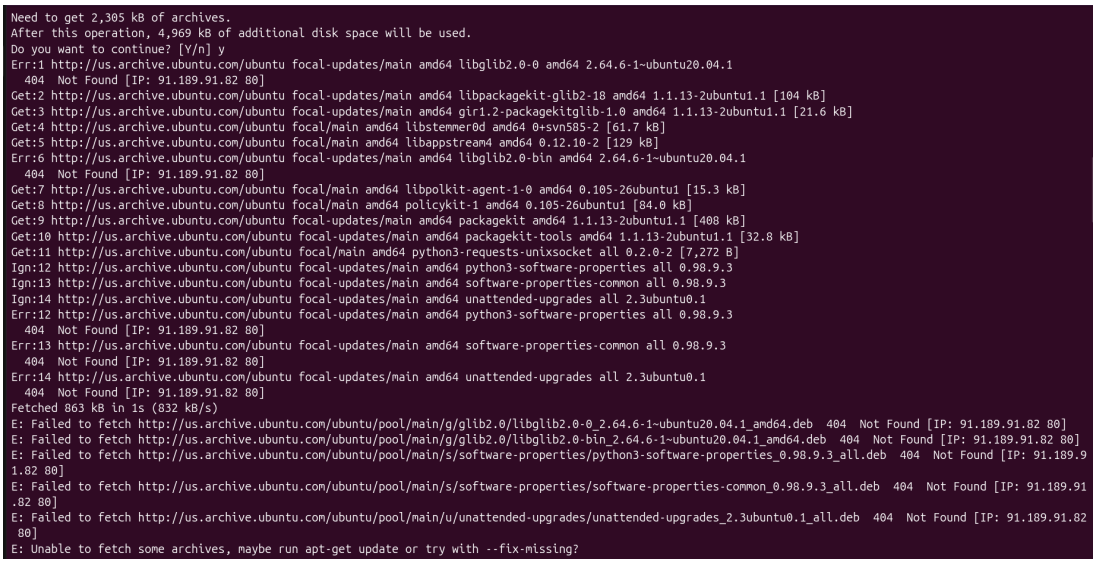


Figure 3: Errors met during the Wireshark installation

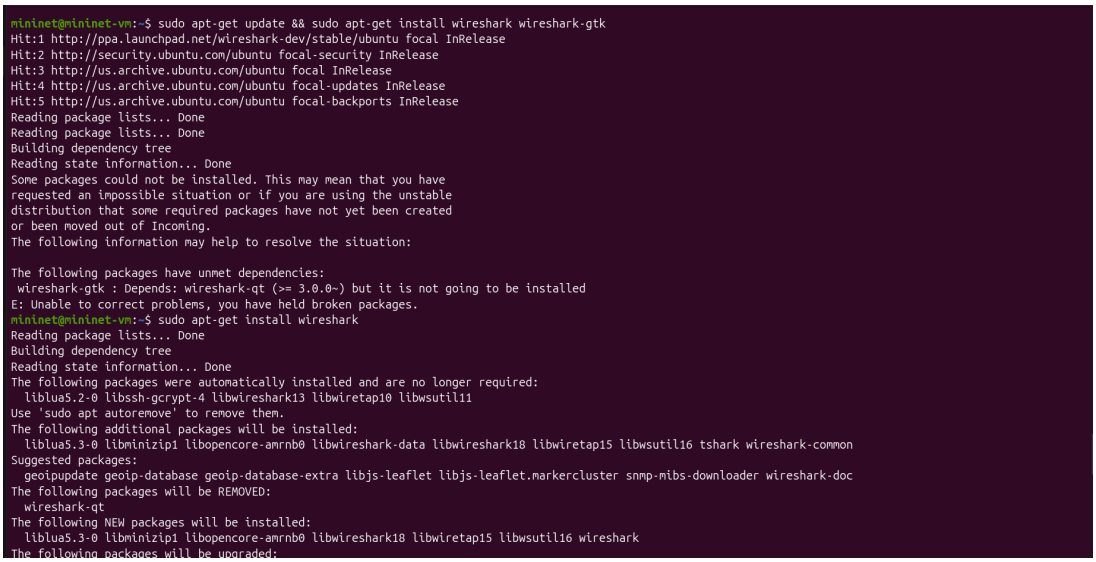


Figure 4: Error2 met during the Wireshark installation

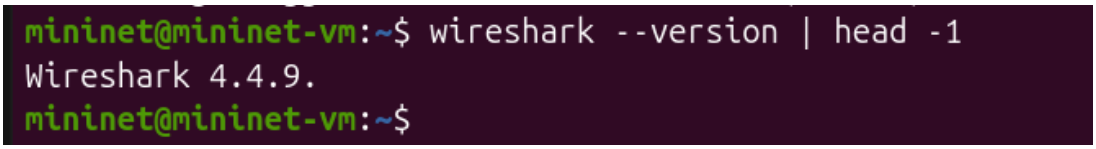


Figure 5: Wireshark 4.4.9 version

2.6 Running Wireshark from the command line

As suggested in the lab statement, Wireshark was started from the SSH session with X11 forwarding, through `xterm`. The command `sudo wireshark` failed with `could not connect to display` and `X11 connection rejected because of wrong authentication`: when running as root through `sudo`, Wireshark does not have access to the X11 authentication cookie of the `mininet` user, stored in `~/.Xauthority`. The problem was solved by passing this file explicitly:

```
xterm &
sudo XAUTHORITY=$HOME/.Xauthority wireshark &
```

The Wireshark window, running in the VM, is then displayed on the host desktop and lists the interfaces of the VM.

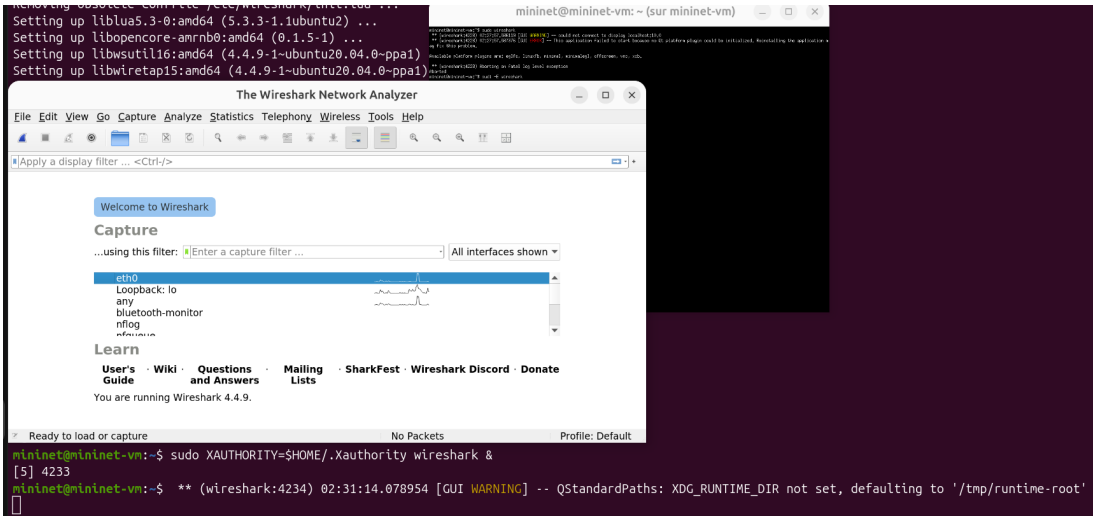


Figure 6: Wireshark displayed on the host through SSH X11 forwarding.

2.7 Desktop installation

The lab statement proposes, as an alternative to the command line, to install a lightweight desktop in the VM (`xinit`, `lxde`, `startx`). This step was not performed: Wireshark already runs correctly through SSH with X11 forwarding , so a desktop environment inside the VM was not necessary.

3 Part B: Experimenting with Mininet

The command `sudo mn -h` displays the startup options of Mininet.

```
mininet@mininet-vm:~$ sudo mn -h
Usage: mn [options]
(type mn -h for details)

The mn utility creates Mininet network from the command line. It can create
parametrized topologies, invoke the Mininet CLI, and run tests.

Options:
  -h, --help            show this help message and exit
  --switch=SWITCH       default|ivs|lxbr|ovs|ovsbr|ovsk|user[,param=value...]
                        user=UserSwitch ovs=OVSSwitch ovsbr=OVSBridge
                        ovsk=OVSSwitch ivs=IVSSwitch lxbr=LinuxBridge
                        default=OVSSwitch
  --host=HOST           cfs|proc|rt[,param=value...] proc=Host
                        rt=CPULimitedHost{'sched': 'rt'}
                        cfs=CPULimitedHost{'sched': 'cfs'}
  --controller=CONTROLLER
                        default|none|nox|ovsc|ref|remote|ryu[,param=value...]
                        ref=Controller ovsc=OVSController nox=NOX
                        remote=RemoteController ryu=Ryu
                        default=DefaultController none=NullController
  --link=LINK           default|ovs|tc|tcu[,param=value...] default=Link
                        tc=TCLink tcu=TCULink ovs=OVSLink
  --topo=TOPO           linear|minimal|reversed|single|torus|tree[,param=value
                        ...] minimal=MinimalTopo linear=LinearTopo
                        reversed=SingleSwitchReversedTopo
                        single=SingleSwitchTopo tree=TreeTopo torus=TorusTopo
  -c, --clean           clean and exit
  --custom=CUSTOM       read custom classes or params from .py file(s)
  --test=TEST           pingall|pingpair|iperf|iperfudp|all|none|build
  -x, --xterms          spawn xterms for each node
  -i IPBASE, --ipbase=IPBASE
                        base IP address for hosts
  --mac                 automatically set host MACs
  --arp                 set all-pairs ARP entries
  -v VERBOSITY, --verbosity=VERBOSITY
                        debug|info|output|warning|warn|error|critical
  --innamespace         sw and ctrl in namespace?
  --listenport=LISTENPORT
                        base port for passive switch listening
  --nolistenport        don't use passive listening port
  --pre=PRE             CLI script to run before tests
  --post=POST           CLI script to run after tests
  --pin                 pin hosts to CPU cores (requires --host cfs or --host
                        rt)
  --nat                 [option=val...] adds a NAT to the topology that
                        connects Mininet hosts to the physical network.
                        Warning: This may route any traffic on the machine
                        that uses Mininet's IP subnet into the Mininet
                        network. If you need to change Mininet's IP subnet,
                        see the --ipbase option.
  --version             prints the version and exits
  -w, --wait            wait for switches to connect
```

Figure 7: Output of `sudo mn -h`.

Wireshark was started on the loopback interface (`lo`) of the VM, since the switch and the controller both run inside the VM and communicate through 127.0.0.1. The display filter `of` given in the lab statement is rejected by Wireshark 4.4 : this short protocol name no longer exists in recent versions. The filter `openflow` is accepted, and, as expected, no OpenFlow packet is displayed since Mininet is not running yet.

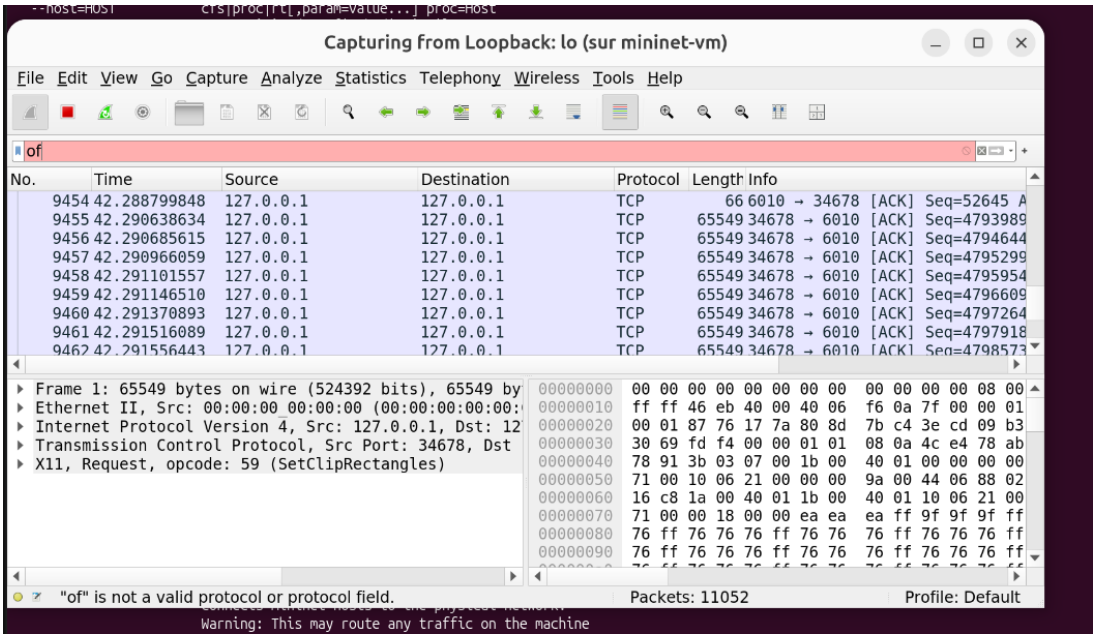


Figure 8: Wireshark on `lo`: filter `of` rejected

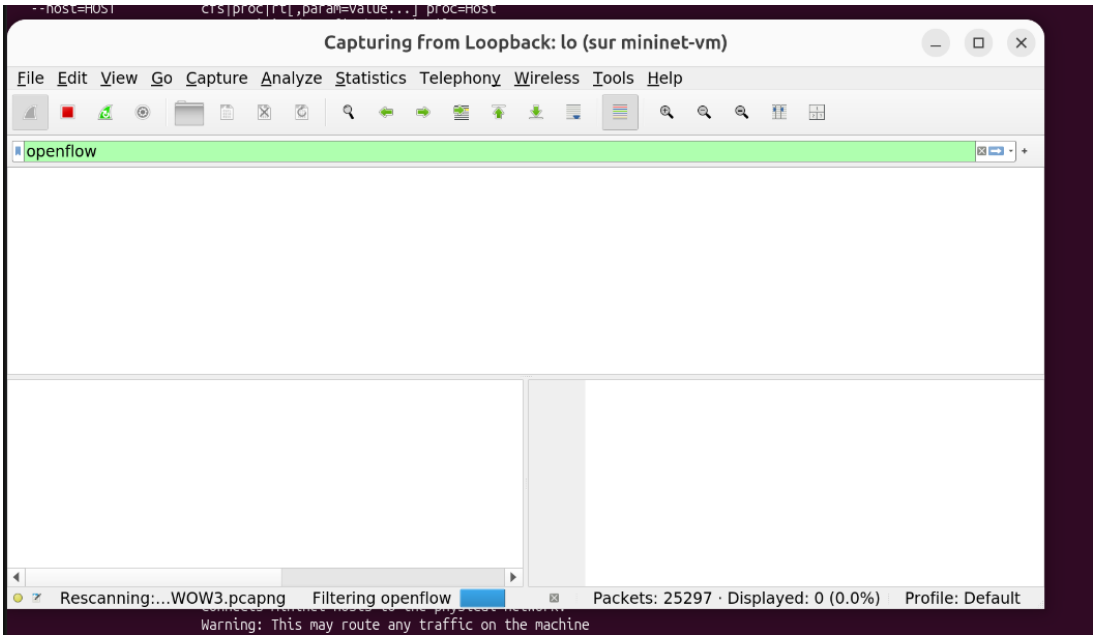


Figure 9: Wireshark on `lo`: filter `openflow` accepted with no packet.

3.1 Part B.1: Interact with Hosts and Switches

Minimal topology. `sudo mn` creates the default minimal topology: one Open vSwitch (`s1`) connected to two hosts (`h1`, `h2`), plus a controller (`c0`). After `exit` and a cleanup with `sudo mn -c`, the topology was started again and explored with the CLI commands `help`, `nodes`, `net` and `dump`. `net` shows the links `h1-eth0` ↔ `s1-eth1` and `h2-eth0` ↔ `s1-eth2`. `dump` shows the IP address and PID of each node, and indicates that the controller listens on `127.0.0.1:6653`, the current standard OpenFlow port.

```
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> help

Documented commands (type help <topic>):
========================================
EOF    gterm  iperfudp  nodes        pingpair      py      switch  xterm
dpctl  help   link      noecho       pingpairfull  quit    time
dump   intfs  links     pingall      ports         sh      wait
exit   iperf  net       pingallfull  px            source  x

You may also send a command to a node using:
  <node> command {args}
For example:
  mininet> h1 ifconfig

The interpreter automatically substitutes IP addresses
for node names when a node is the first arg, so commands
like
  mininet> h2 ping h3
should work.

Some character-oriented interactive commands require
noecho:
  mininet> noecho h2 vi foo.py
However, starting up an xterm/gterm is generally better:
  mininet> xterm h2
```

Figure 10: Starting the default minimal topology with `sudo mn` and listing the Mininet CLI commands with `help`.

```
mininet> nodes
available nodes are:
c0 h1 h2 s1
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0
c0
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=4520>
<Host h2: h2-eth0:10.0.0.2 pid=4522>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None pid=4527>
<Controller c0: 127.0.0.1:6653 pid=4513>
mininet> h1 config -a
bash: config: command not found
mininet> h1 ifconfig -a
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.1  netmask 255.0.0.0  broadcast 10.255.255.255
        ether 8a:1e:3e:7f:dd:50  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

mininet> s1 ifconfig -a
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        ether 08:00:27:6d:08:27  txqueuelen 1000  (Ethernet)
        RX packets 2346146  bytes 198097042 (198.0 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 2314798  bytes 3256266378 (3.2 GB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 11: Output of `help`, `nodes`, `net` and `dump`.

```
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 84463  bytes 3129217379 (3.1 GB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 84463  bytes 3129217379 (3.1 GB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

ovs-system: flags=4098<BROADCAST,MULTICAST>  mtu 1500
        ether f2:56:7d:9d:62:95  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

s1: flags=4098<BROADCAST,MULTICAST>  mtu 1500
        ether 16:d9:20:49:89:43  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

s1-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        ether f6:eb:a8:e4:d8:91  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

s1-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        ether ba:8c:0b:09:03:f6  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

mininet> h1 ps -a
    PID TTY          TIME CMD
    654 tty1     00:00:00 bash
   4218 pts/0    00:00:00 xterm
   4507 pts/2    00:00:00 sudo
```

Figure 12: s1 ifconfig -a: interfaces of s1 (root namespace).



Figure 13: ps -a on h1 and s1: same processes, then first ping.

Network namespaces. `h1 ifconfig -a` only shows `h1-eth0` (10.0.0.1/8) and the loopback interface: the host runs in its own network namespace, so its interfaces are not visible from the rest of the system. On the contrary, `s1 ifconfig -a` shows all the interfaces of the VM: the switch ports `s1-eth1` and `s1-eth2`, the Open vSwitch interfaces `s1` and `ovs-system`, and the VM connection `eth0` (10.0.2.15), because the switch runs in the root network namespace.

```
s1: flags=4098<BROADCAST,MULTICAST>  mtu 1500
        ether 16:d9:20:49:89:43  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

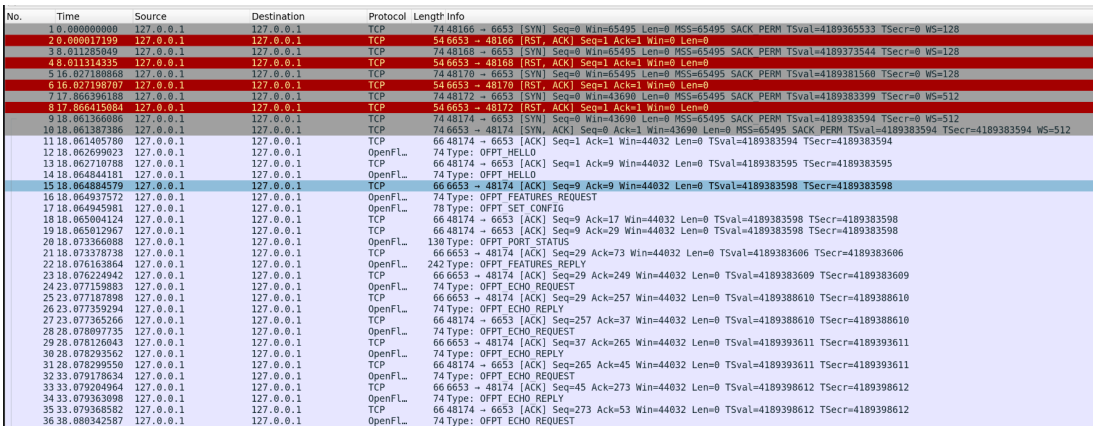
s1-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        ether f6:eb:a8:e4:d8:91  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

s1-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        ether ba:8c:0b:09:03:f6  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

mininet> h1 ps -a
    PID TTY          TIME CMD
    654 tty1     00:00:00 bash
   4218 pts/0    00:00:00 xterm
   4507 pts/2    00:00:00 sudo
   4508 pts/2    00:00:00 mn
   4555 pts/3    00:00:00 controller
   4585 pts/4    00:00:00 ps
mininet> s1 ps -a
    PID TTY          TIME CMD
    654 tty1     00:00:00 bash
   4218 pts/0    00:00:00 xterm
   4507 pts/2    00:00:00 sudo
   4508 pts/2    00:00:00 mn
   4555 pts/3    00:00:00 controller
   4587 pts/6    00:00:00 ps
mininet> h1 ping -c 1 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=3.89 ms
```

Figure 14: `ifconfig -a` on h1 and on s1.

First ping and OpenFlow messages. To observe the full sequence described in the lab, Wireshark was capturing on `lo` with the capture filter `tcp port 6653 or tcp port 6633`, and Mininet was freshly started . The switch and the controller use OpenFlow 1.0. At startup, the switch first tries to connect to the controller before it listens (TCP `SYN` answered by `RST`), then the TCP connection is established and the OpenFlow handshake takes place: `HELLO`, `FEATURES_REQUEST`, `SET_CONFIG`, `PORT_STATUS` and `FEATURES_REPLY`. `ECHO_REQUEST/REPLY` messages are then exchanged every 5 seconds as a keepalive.



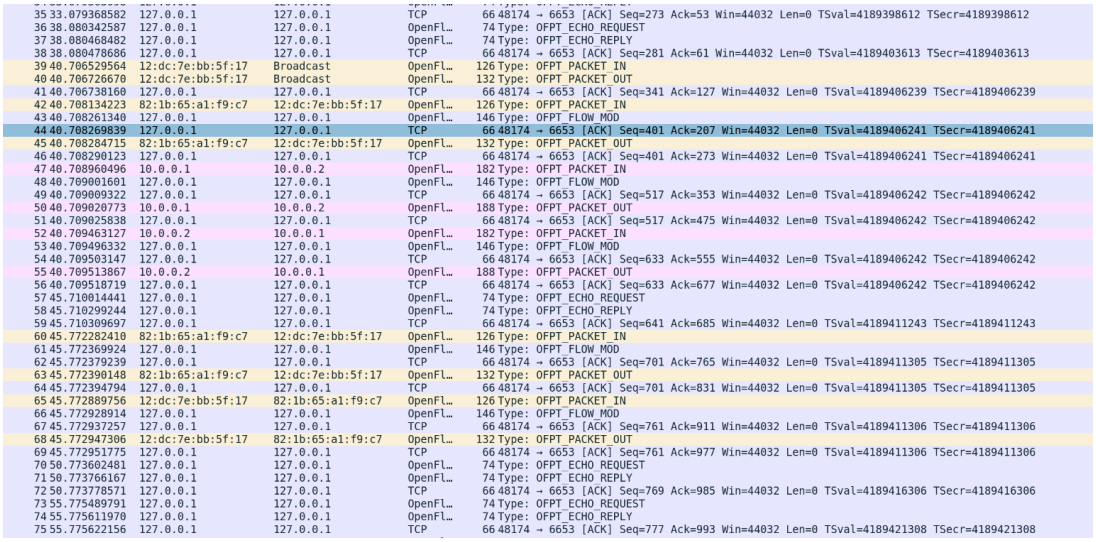
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	48166 → 6653 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=4189365533 TSecr=0 WS=128
2	0.000017199	127.0.0.1	127.0.0.1	TCP	54	6653 → 48166 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3	0.011285949	127.0.0.1	127.0.0.1	TCP	74	48168 → 6653 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=4189373544 TSecr=0 WS=128
4	8.011314335	127.0.0.1	127.0.0.1	TCP	54	6653 → 48168 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5	16.027180868	127.0.0.1	127.0.0.1	TCP	74	48170 → 6653 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=4189381560 TSecr=0 WS=128
6	16.027198707	127.0.0.1	127.0.0.1	TCP	54	6653 → 48170 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7	17.866395188	127.0.0.1	127.0.0.1	TCP	74	48172 → 6653 [SYN] Seq=0 Win=43690 Len=0 MSS=65495 SACK_PERM TSval=4189383399 TSecr=0 WS=512
8	17.866415084	127.0.0.1	127.0.0.1	TCP	54	6653 → 48172 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
9	18.061366086	127.0.0.1	127.0.0.1	TCP	74	48174 → 6653 [SYN] Seq=0 Win=43690 Len=0 MSS=65495 SACK_PERM TSval=4189383594 TSecr=0 WS=512
10	18.061387386	127.0.0.1	127.0.0.1	TCP	74	6653 → 48174 [SYN, ACK] Seq=0 Ack=1 Win=43690 Len=0 MSS=65495 SACK_PERM TSval=4189383594 TSecr=4189383594 WS=512
11	18.061405780	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=1 Ack=1 Win=44032 Len=0 TSval=4189383594 TSecr=4189383594
12	18.062699023	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_HELLO
13	18.062710788	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=1 Ack=9 Win=44032 Len=0 TSval=4189383595 TSecr=4189383595
14	18.064844181	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_HELLO
15	18.064884579	127.0.0.1	127.0.0.1	TCP	66	6653 → 48174 [ACK] Seq=9 Ack=9 Win=44032 Len=0 TSval=4189383598 TSecr=4189383598
16	18.064937572	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_FEATURES_REQUEST
17	18.064945981	127.0.0.1	127.0.0.1	OpenFl…	78	Type: OFPT_SET_CONFIG
18	18.065004124	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=9 Ack=17 Win=44032 Len=0 TSval=4189383598 TSecr=4189383598
19	18.065012967	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=9 Ack=29 Win=44032 Len=0 TSval=4189383598 TSecr=4189383598
20	18.073366088	127.0.0.1	127.0.0.1	OpenFl…	130	Type: OFPT_PORT_STATUS
21	18.073378738	127.0.0.1	127.0.0.1	TCP	66	6653 → 48174 [ACK] Seq=29 Ack=73 Win=44032 Len=0 TSval=4189383606 TSecr=4189383606
22	18.076163864	127.0.0.1	127.0.0.1	OpenFl…	242	Type: OFPT_FEATURES_REPLY
23	18.076224942	127.0.0.1	127.0.0.1	TCP	66	6653 → 48174 [ACK] Seq=29 Ack=249 Win=44032 Len=0 TSval=4189383609 TSecr=4189383609
24	23.077159883	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REQUEST
25	23.077187898	127.0.0.1	127.0.0.1	TCP	66	6653 → 48174 [ACK] Seq=29 Ack=257 Win=44032 Len=0 TSval=4189388610 TSecr=4189388610
26	23.077359294	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REPLY
27	23.077365266	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=257 Ack=37 Win=44032 Len=0 TSval=4189388610 TSecr=4189388610
28	28.078097735	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REQUEST
29	28.078126043	127.0.0.1	127.0.0.1	TCP	66	6653 → 48174 [ACK] Seq=37 Ack=265 Win=44032 Len=0 TSval=4189393611 TSecr=4189393611
30	28.078293562	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REPLY
31	28.078299550	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=265 Ack=45 Win=44032 Len=0 TSval=4189393611 TSecr=4189393611
32	33.079178634	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REQUEST
33	33.079204964	127.0.0.1	127.0.0.1	TCP	66	6653 → 48174 [ACK] Seq=45 Ack=273 Win=44032 Len=0 TSval=4189398612 TSecr=4189398612
34	33.079363098	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REPLY
35	33.079368582	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=273 Ack=53 Win=44032 Len=0 TSval=4189398612 TSecr=4189398612
36	38.080342587	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REQUEST

Figure 15: Connection setup and OpenFlow handshake between s1 and the controller.

During the first ping, the sequence is exactly the one described in the lab:

- h1 sends an ARP request in broadcast: the switch has no matching rule and sends it to the controller (`PACKET_IN`); the controller answers with a `PACKET_OUT` that floods it to the other ports.
- h2 answers with an ARP reply, which also goes to the controller (`PACKET_IN`); the controller installs a rule (`FLOW_MOD`) and sends the reply to h1 (`PACKET_OUT`).
- The ICMP Echo Request and Echo Reply follow the same process (`PACKET_IN`, `FLOW_MOD`, `PACKET_OUT`).

About 5 seconds later, unicast ARP messages are exchanged in both directions: Linux checks that the MAC address of the other host is still valid.



35	33.079368582	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=273 Ack=53 Win=44032 Len=0 TSval=4189398612 TSecr=4189398612
36	38.080342587	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REQUEST
37	38.080468482	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REPLY
38	38.080478686	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=281 Ack=61 Win=44032 Len=0 TSval=4189403613 TSecr=4189403613
39	40.706529564	12:dc:7e:bb:5f:17	Broadcast	OpenFl…	126	Type: OFPT_PACKET_IN
40	40.706726670	12:dc:7e:bb:5f:17	Broadcast	OpenFl…	132	Type: OFPT_PACKET_OUT
41	40.706738160	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=341 Ack=127 Win=44032 Len=0 TSval=4189406239 TSecr=4189406239
42	40.708134223	82:1b:65:a1:f9:c7	12:dc:7e:bb:5f:17	OpenFl…	126	Type: OFPT_PACKET_IN
43	40.708261340	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
44	40.708269839	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=401 Ack=207 Win=44032 Len=0 TSval=4189406241 TSecr=4189406241
45	40.708284715	82:1b:65:a1:f9:c7	12:dc:7e:bb:5f:17	OpenFl…	132	Type: OFPT_PACKET_OUT
46	40.708290123	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=401 Ack=273 Win=44032 Len=0 TSval=4189406241 TSecr=4189406241
47	40.708960496	10.0.0.1	10.0.0.2	OpenFl…	182	Type: OFPT_PACKET_IN
48	40.709001601	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
49	40.709009322	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=517 Ack=353 Win=44032 Len=0 TSval=4189406242 TSecr=4189406242
50	40.709020773	10.0.0.1	10.0.0.2	OpenFl…	188	Type: OFPT_PACKET_OUT
51	40.709025838	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=517 Ack=475 Win=44032 Len=0 TSval=4189406242 TSecr=4189406242
52	40.709463127	10.0.0.2	10.0.0.1	OpenFl…	182	Type: OFPT_PACKET_IN
53	40.709496332	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
54	40.709503147	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=633 Ack=555 Win=44032 Len=0 TSval=4189406242 TSecr=4189406242
55	40.709513867	10.0.0.2	10.0.0.1	OpenFl…	188	Type: OFPT_PACKET_OUT
56	40.709518719	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=633 Ack=677 Win=44032 Len=0 TSval=4189406242 TSecr=4189406242
57	45.710014441	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REQUEST
58	45.710299244	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REPLY
59	45.710309697	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=641 Ack=685 Win=44032 Len=0 TSval=4189411243 TSecr=4189411243
60	45.772282410	82:1b:65:a1:f9:c7	12:dc:7e:bb:5f:17	OpenFl…	126	Type: OFPT_PACKET_IN
61	45.772369924	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
62	45.772379239	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=701 Ack=765 Win=44032 Len=0 TSval=4189411305 TSecr=4189411305
63	45.772390148	82:1b:65:a1:f9:c7	12:dc:7e:bb:5f:17	OpenFl…	132	Type: OFPT_PACKET_OUT
64	45.772394794	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=701 Ack=831 Win=44032 Len=0 TSval=4189411305 TSecr=4189411305
65	45.772889756	12:dc:7e:bb:5f:17	82:1b:65:a1:f9:c7	OpenFl…	126	Type: OFPT_PACKET_IN
66	45.772928914	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
67	45.772937257	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=761 Ack=911 Win=44032 Len=0 TSval=4189411306 TSecr=4189411306
68	45.772947306	12:dc:7e:bb:5f:17	82:1b:65:a1:f9:c7	OpenFl…	132	Type: OFPT_PACKET_OUT
69	45.772951775	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=761 Ack=977 Win=44032 Len=0 TSval=4189411306 TSecr=4189411306
70	50.773602481	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REQUEST
71	50.773766167	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REPLY
72	50.773778571	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=769 Ack=985 Win=44032 Len=0 TSval=4189416306 TSecr=4189416306
73	55.775489791	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REQUEST
74	55.775611970	127.0.0.1	127.0.0.1	OpenFl…	74	Type: OFPT_ECHO_REPLY
75	55.775622156	127.0.0.1	127.0.0.1	TCP	66	48174 → 6653 [ACK] Seq=777 Ack=993 Win=44032 Len=0 TSval=4189421308 TSecr=4189421308

Figure 16: `PACKET_IN`, `PACKET_OUT` and `FLOW_MOD`.

This Figure 17 shows the details of the first `PACKET_IN` message (OpenFlow 1.0). The *Reason* field (`No matching flow`) shows that the packet is sent to the controller because no rule matches it. The packet arrived on port 1 of the switch (`s1-eth1`, connected to h1). The *Buffer Id* `0xffffffff` means that the switch did not buffer the packet and sends it entirely to the controller (42 bytes). The encapsulated frame is the ARP request sent by h1 to the broadcast address `ff:ff:ff:ff:ff:ff`.



Figure 17: Details of the first `PACKET_IN` (OpenFlow 1.0): ARP request from h1 in broadcast.

Ping times. The first ping took 3.52 ms, while the second one took only 0.309 ms (about 11 times faster). During the second ping, no OpenFlow message was generated: the rules installed during the first ping were used directly by the switch. The second RTT is not yet below 100 µs, as announced in the lab, because of the two-level architecture of Open vSwitch: OpenFlow rules are stored in the user-space daemon `ovs-vswitchd`, while the kernel only keeps a cache of active flows. The first packet of a new flow is therefore still processed by `ovs-vswitchd` (without involving the controller), and the following packets are handled directly by the kernel. This is confirmed with `h1 ping -c 5 h2` (Table 1): the RTT decreases progressively and the last two packets are below 100 µs.

```
rm -f ~/.ssh/mn/*
*** Cleanup complete.
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ping -c 1 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=3.52 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 3.516/3.516/3.516/0.000 ms
mininet> h1 ping -c 1 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.309 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.309/0.309/0.309/0.000 ms
mininet>
```

Figure 18: Round-trip times of the first and second ping.

Packet	RTT	Explanation
1	1.28 ms	rules expired (`idle_timeout`): packet sent to the controller
2–3	0.726 / 0.301 ms	rule installed, processed by `ovs-vswitchd`
4–5	0.089 / 0.090 ms	processed directly by the kernel cache

Table 1: Round-trip times of `h1 ping -c 5 h2`.

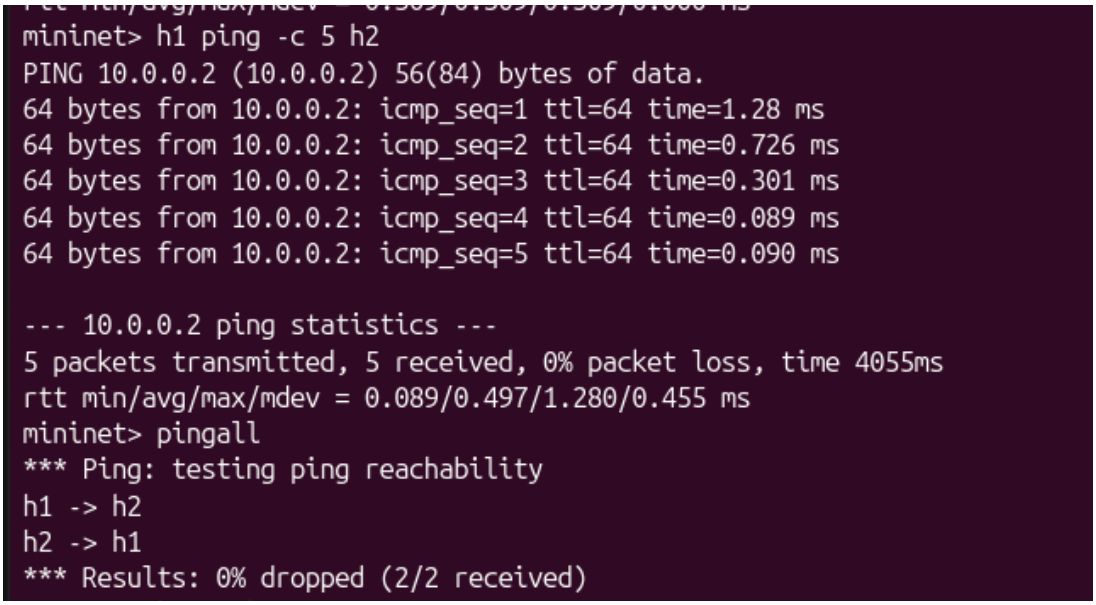


Figure 19: `h1 ping -c 5 h2`: RTT below 100 µs for the last packets.

pingall. The built-in `pingall` command performs an all-pairs ping test: both pings (h1 → h2 and h2 → h1) succeeded

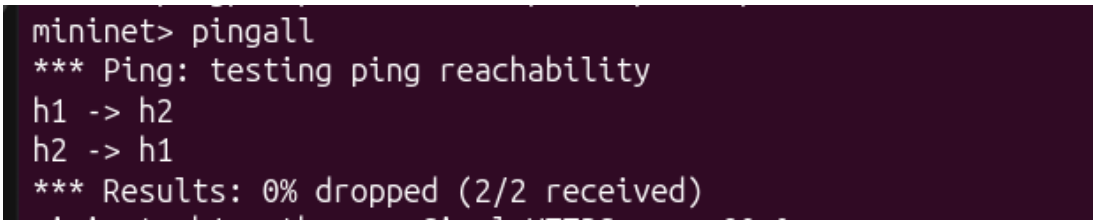


Figure 20: pingall

HTTP server. Mininet hosts can run any application available on the VM. The command of the lab statement, `h1 python -m SimpleHTTPServer 80 &`, failed with `No module named SimpleHTTPServer`: in the VM, `python` is Python 3, in which this Python 2 module was renamed `http.server`. Since no server was running, the request from h2 failed with `Connection refused`, which shows that h1 is reachable but that no service listens on port 80. With the Python 3 equivalent, h2 successfully downloaded the page served by h1 (HTTP `200 OK`), and the server log on h1 shows the request coming from 10.0.0.2:

```
mininet> h1 python3 -m http.server 80 &
mininet> h2 wget -O - h1
mininet> h1 kill %python3
```



Figure 21: Command of the lab statement: `SimpleHTTPServer` not available in Python 3.



Figure 22: HTTP server on h1 and request from h2.



Figure 23: Server log on h1: request from 10.0.0.2.

3.2 Part B.2: Changing Topology Size and Type

The `-topo` option allows starting Mininet with another topology than the default one, with parameters.

Question 1: How does this topology look like? The `single,3` topology is a star topology: a single OpenFlow switch (s1) connected to three hosts (h1, h2, h3), each on its own switch port (`s1-eth1`, `s1-eth2`, `s1-eth3`), as shown by the `net` command. The `pingall` regression test performed $3 \times 2 = 6$ pings (each host pings the two others) with 0% packet loss.

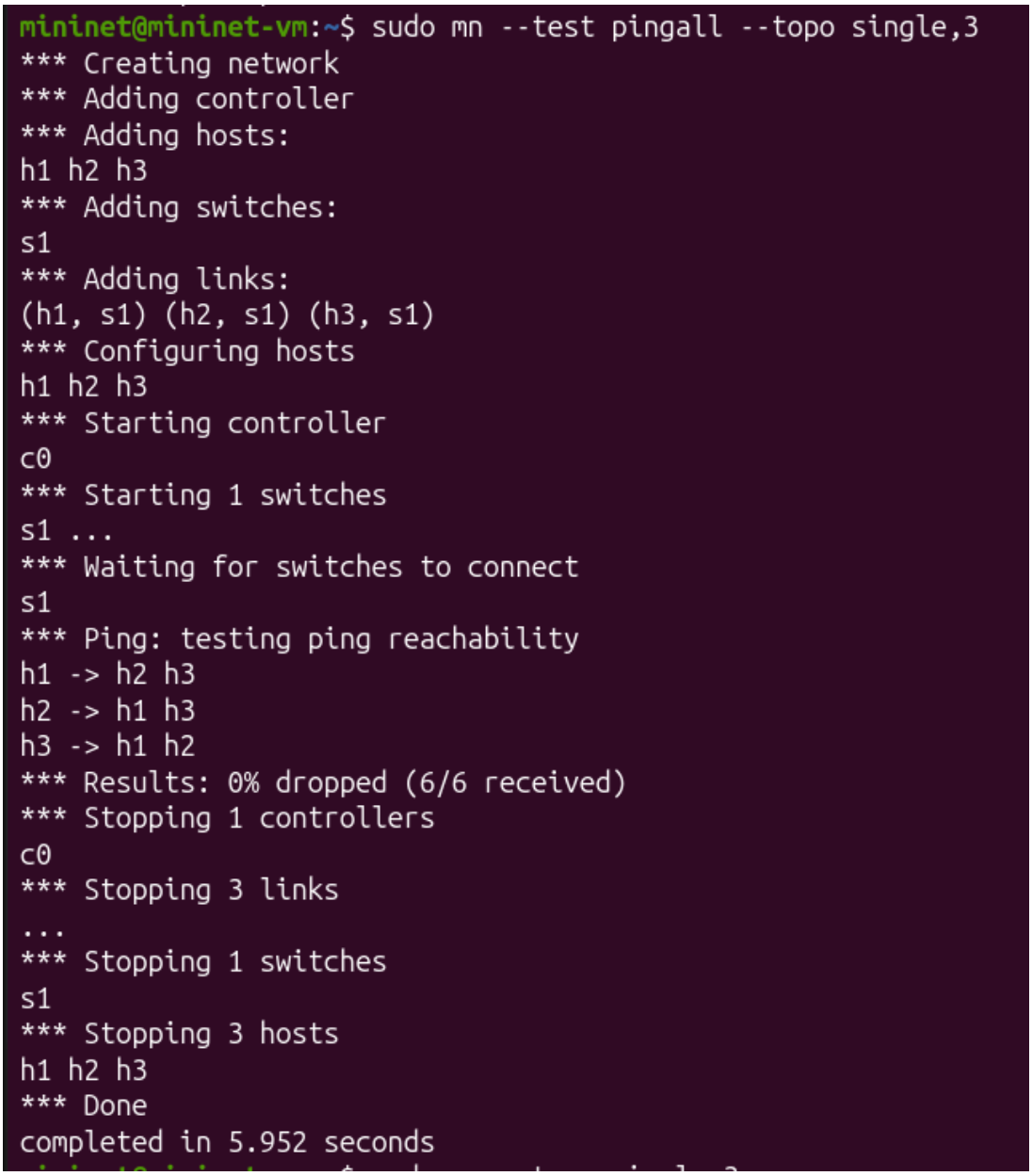


Figure 24: `sudo mn test pingall topo single3` .

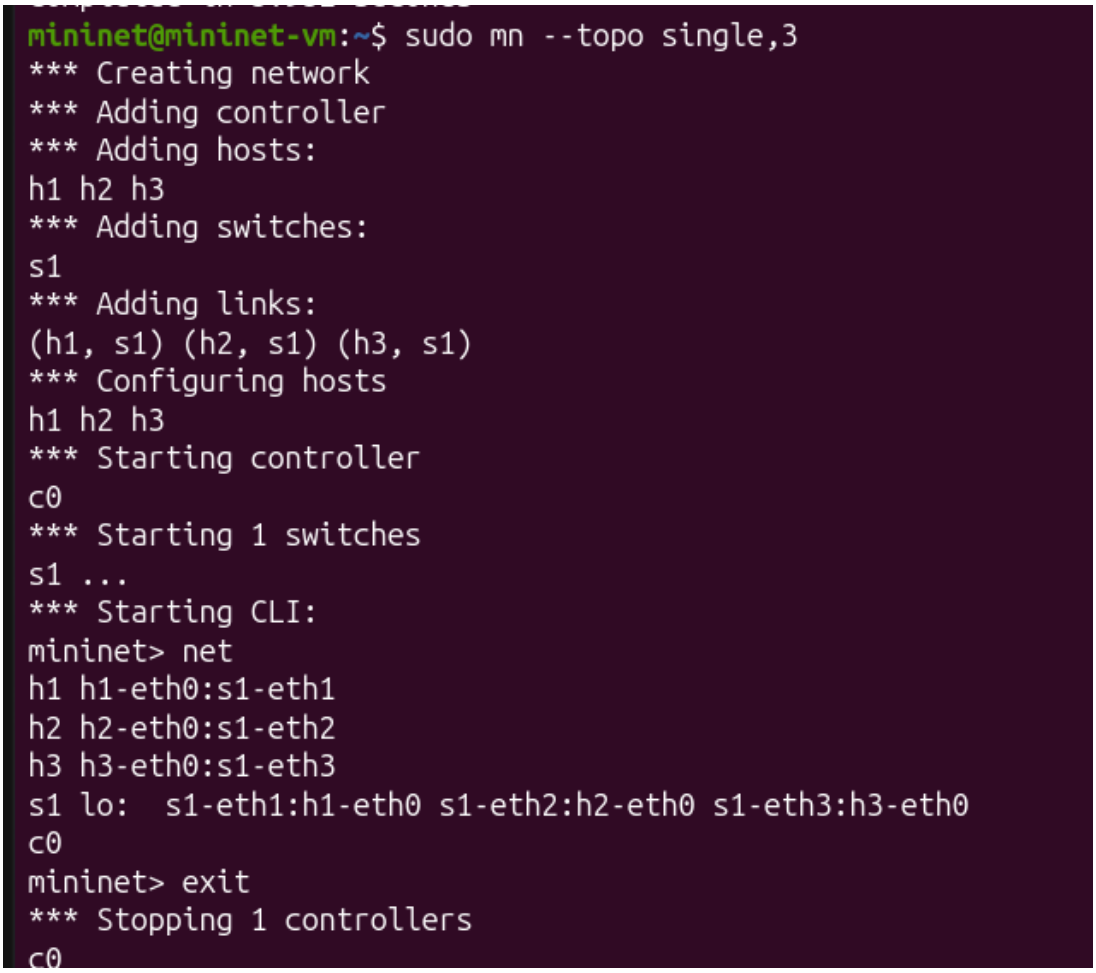


Figure 25: `topo single,3` and `net`.

Question 2: How does this topology look like? The `linear,4` topology consists of four switches connected in a chain (s1–s2–s3–s4), each switch having one host attached (h1 to h4). It contains 7 links: 4 host–switch links and 3 switch–switch links. As shown by `net`, each host is connected to port `eth1` of its switch, while the middle switches (s2, s3) use three ports: one towards their host, one towards the previous switch and one towards the next one. The `pingall` test performed $4 \times 3 = 12$ pings with 0% packet loss.

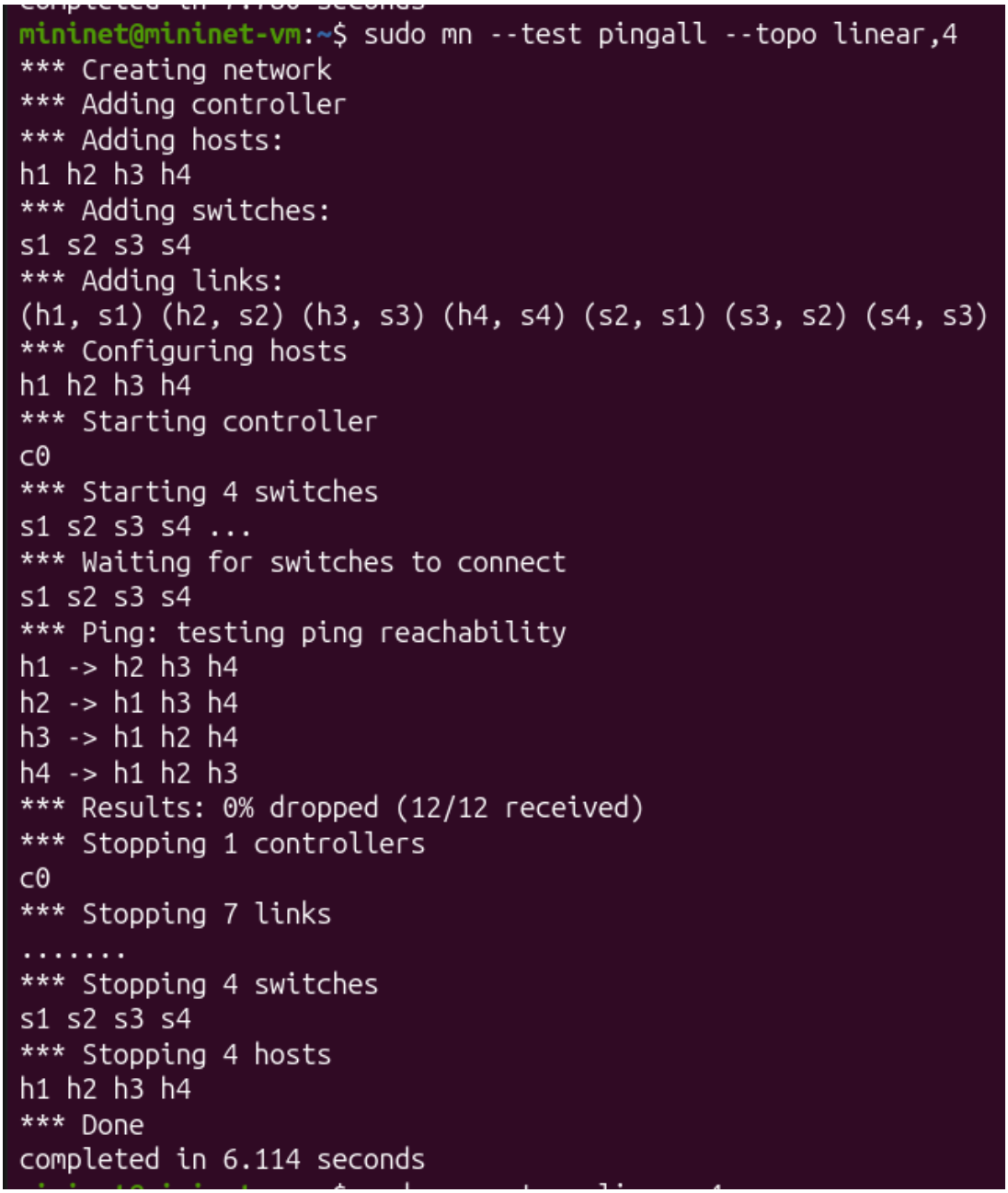


Figure 26: `sudo mn -test pingall -topo linear4`.

```
mininet@mininet-vm:~$ sudo mn --topo linear,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s2, s1) (s3, s2) (s4, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth1
h3 h3-eth0:s3-eth1
h4 h4-eth0:s4-eth1
s1 lo:  s1-eth1:h1-eth0 s1-eth2:s2-eth2
s2 lo:  s2-eth1:h2-eth0 s2-eth2:s1-eth2 s2-eth3:s3-eth2
s3 lo:  s3-eth1:h3-eth0 s3-eth2:s2-eth3 s3-eth3:s4-eth2
s4 lo:  s4-eth1:h4-eth0 s4-eth2:s3-eth3
c0
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 7 links
.......
*** Stopping 4 switches
s1 s2 s3 s4
*** Stopping 4 hosts
h1 h2 h3 h4
*** Done
completed in 7.166 seconds
```

Figure 27: `sudo mn -topo linear4`.

Question 3: What are the different types of possible topologies in Mininet? According to the `-topo` section of `mn -h` (Figure 7), Mininet provides the following built-in parameterized topologies:

- **minimal**: one switch connected to two hosts (default topology).
- **single,N**: one switch connected to N hosts (star topology).
- **reversed,N**: same as single, but the hosts are connected to the switch ports in reverse order.

- **linear,K[,N]**: K switches connected in a chain, with N hosts per switch.
- **tree,depth=D,fanout=F**: a tree of switches of depth D, each switch having F children, the hosts are attached to the leaf switches (F^D hosts in total).
- **torus,X,Y**: an $X \times Y$ grid of switches whose edges wrap around, unlike the other topologies, it contains loops.

3.3 Part B.3: Custom Topologies

Custom topologies can be defined with Mininet's Python API [6]. The example of the lab statement, `custom/topo-2sw-2host.py`, defines a class inheriting from `Topo` whose `build()` method creates two hosts (h1, h2) and two switches (s3, s4) with `addHost`, `addSwitch` and `addLink` (h1–s3–s4–h2). The `topos` dictionary makes this topology available on the command line with `-topo mytopo`. The test reported 0% packet loss.

```
mininet@mininet-vm:~$ sudo mn --custom ~/mininet/custom/topo-2sw-2host.py --topo mytopo --test pingall
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s3 s4
*** Adding links:
(h1, s3) (s3, s4) (s4, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
s3 s4 ...
*** Waiting for switches to connect
s3 s4
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
*** Stopping 1 controllers
c0
*** Stopping 3 links
...
*** Stopping 2 switches
s3 s4
*** Stopping 2 hosts
h1 h2
*** Done
completed in 5.518 seconds
```

Figure 28: Example of the lab statement: `topo-2sw-2host.py` and `pingall`.

Following the same structure, the following script creates the topology of Figure 2 of the lab statement: two switches (s1, s2), hosts h3 and h4 in subnet 10.0.1.0/24 on s1, host h5 in subnet 10.0.2.0/24 on s2, and a controller c0 listening on 127.0.0.1:6633.

```python
from mininet.net import Mininet
from mininet.topo import Topo
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel


class Fig2Topo(Topo):
    def build(self)
        s1 = self.addSwitch('s1')
        s2 = self.addSwitch('s2')
        h3 = self.addHost('h3', ip='10.0.1.2/24')
        h4 = self.addHost('h4', ip='10.0.1.3/24')
        h5 = self.addHost('h5', ip='10.0.2.2/24')
        self.addLink(h3, s1)
        self.addLink(h4, s1)
        self.addLink(s1, s2)
        self.addLink(s2, h5)

topos = {'fig2': Fig2Topo}


if __name__ == '__main__':
    setLogLevel('info')
    net = Mininet(topo=Fig2Topo(), controller=None)
    net.addController('c0', controller=Controller, ip='127.0.0.1', port=6633)
    net.start()

    for host in net.hosts:
        host.cmd(f'ip route add default dev {host.name}-eth0')

    net.pingAll()
    CLI(net)
    net.stop()
```

Figure 29: script to create the topology

Results. When the script is used as a custom topology file, as in the lab statement (`sudo mn -custom topo_fig2.py -topo fig2 -test pingall`), only h3 and h4 can reach each other (66% dropped, 2/6 received). Hosts h3/h4 and h5 belong to two different IP subnets, and each host only has a route to its own subnet. Since the switches only perform layer-2 forwarding and there is no router in the topology, the addresses 10.0.1.1 and 10.0.2.1 shown on the switches in Figure 2 act as subnet gateways that do not exist in Mininet.

To solve this problem, the script adds a default route pointing to the interface of each host (`ip route add default dev hX-eth0`). Each host then sends ARP requests directly for any destination, and since s1 and s2 form a single layer-2 network, all hosts can reach each other. Running the script directly (`sudo python3 topo_fig2.py`) gives 0% packet loss (6/6 received). The routing table of h3 shows the added default route.

```
mininet@mininet-vm:~/lab1$ ls -l topo_fig2.py
-rw-rw-r-- 1 mininet mininet 1304 Oct 14 00:46 topo_fig2.py
mininet@mininet-vm:~/lab1$ sudo mn --custom topo_fig2.py --topo fig2 --test pingall
*** Creating network
*** Adding controller
*** Adding hosts:
h3 h4 h5
*** Adding switches:
s1 s2
*** Adding links:
(h3, s1) (h4, s1) (s1, s2) (s2, h5)
*** Configuring hosts
h3 h4 h5
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...
*** Waiting for switches to connect
s1 s2
*** Ping: testing ping reachability
h3 -> h4 X
h4 -> h3 X
h5 -> X X
*** Results: 66% dropped (2/6 received)
*** Stopping 1 controllers
c0
*** Stopping 4 links
....
*** Stopping 2 switches
s1 s2
*** Stopping 3 hosts
h3 h4 h5
*** Done
completed in 5.519 seconds
```

Figure 30: Custom topology loaded with `-custom`: 66% dropped.

```
mininet@mininet-vm:~/lab1$ sudo python3 topo_fig2.py
*** Creating network
*** Adding hosts:
h3 h4 h5
*** Adding switches:
s1 s2
*** Adding links:
(h3, s1) (h4, s1) (s1, s2) (s2, h5)
*** Configuring hosts
h3 h4 h5
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...
*** Ping: testing ping reachability
h3 -> h4 h5
h4 -> h3 h5
h5 -> h3 h4
*** Results: 0% dropped (6/6 received)
*** Starting CLI:
mininet> net
h3 h3-eth0:s1-eth1
h4 h4-eth0:s1-eth2
h5 h5-eth0:s2-eth2
s1 lo:  s1-eth1:h3-eth0 s1-eth2:h4-eth0 s1-eth3:s2-eth1
s2 lo:  s2-eth1:s1-eth3 s2-eth2:h5-eth0
c0
mininet>  h3 route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         0.0.0.0         0.0.0.0         U     0      0        0 h3-eth0
10.0.1.0        0.0.0.0         255.255.255.0   U     0      0        0 h3-eth0
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 4 links
....
*** Stopping 2 switches
s1 s2
*** Stopping 3 hosts
h3 h4 h5
*** Done
```

Figure 31: Script with default routes: 0% dropped, `net` and routing table of h3.

3.4 Part B.4: ID = MAC

By default, Mininet assigns random MAC addresses to the hosts. Starting Mininet twice with `sudo mn` gave two different MAC addresses for h1 (`f6:02:90:7e:a8:2f`, then `de:3e:ad:bc:61:31`). Since the addresses change at every start, it is difficult to correlate the control traffic captured in Wireshark with a specific host.

With the `-mac` option, the MAC addresses are derived from the host number and become small, unique and easy to read: `00:00:00:00:00:01` for h1 and `00:00:00:00:00:02` for h2. The IP addresses (10.0.0.1 and 10.0.0.2) also follow the host number.

```
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.1  netmask 255.0.0.0  broadcast 10.255.255.255
        ether f6:02:90:7e:a8:2f  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
```

Figure 32: Before: random MAC address of h1, different at each start of Mininet.

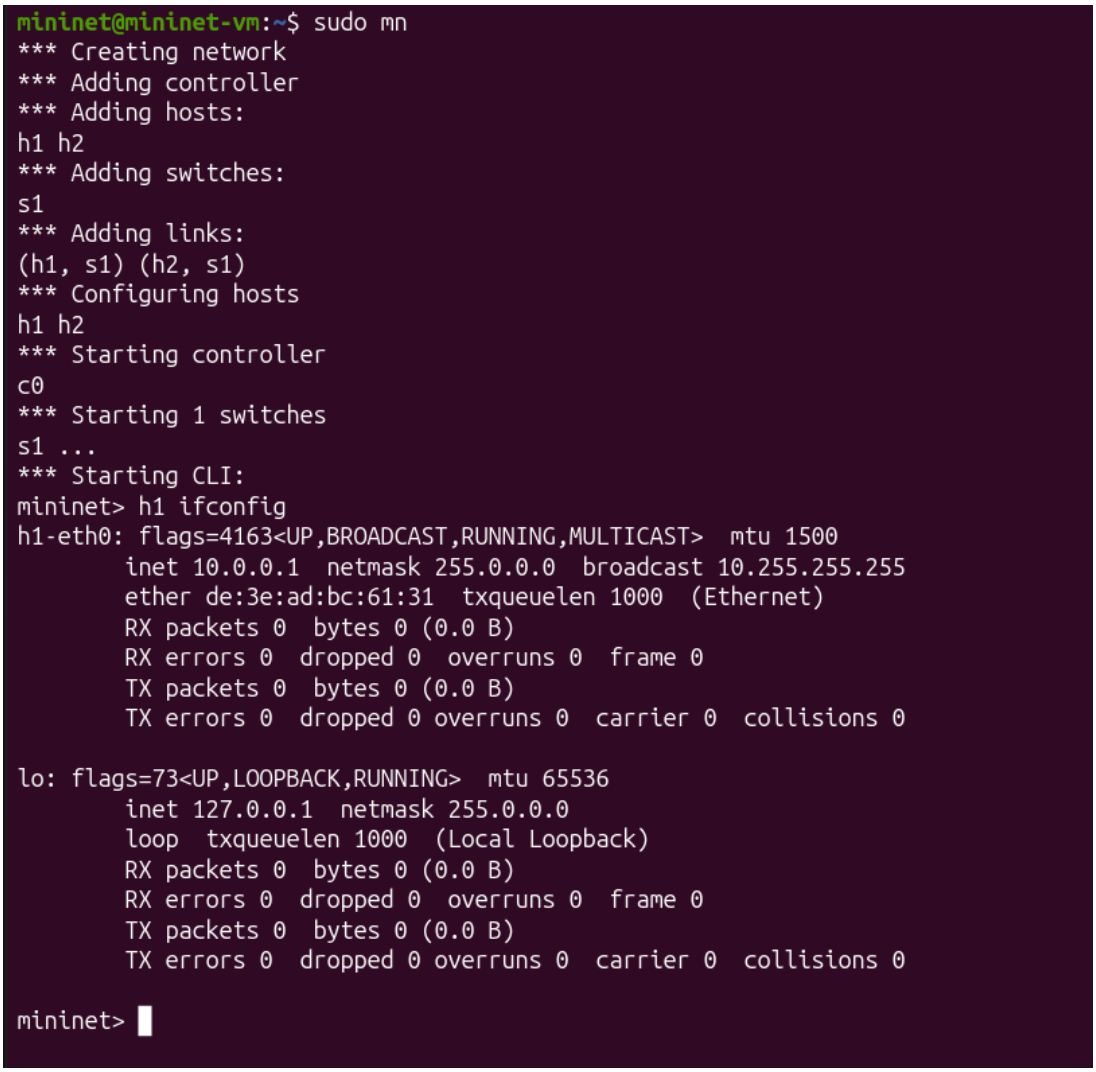


Figure 33: Before: random MAC address of h1, different at each start of Mininet.

```
mininet@mininet-vm:~$ sudo mn --mac
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.1  netmask 255.0.0.0  broadcast 10.255.255.255
        ether 00:00:00:00:00:01  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 34: After: `sudo mn -mac`, MAC addresse h1.

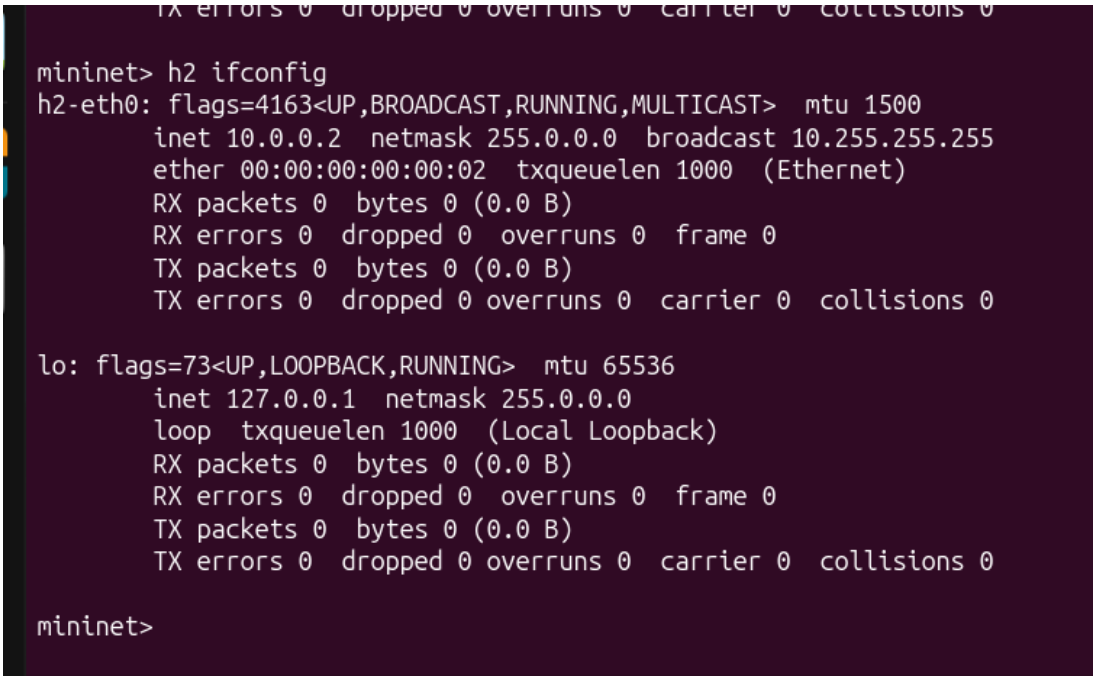


Figure 35: After: `sudo mn -mac`, MAC addresse h2.

3.5 Part B.5: Links Variation

Link parameters from the command line. Mininet can limit the bandwidth and add delay on links with the `-link tc` option, which uses Linux Traffic Control (`tc`):

```
sudo mn --link tc,bw=10,delay=10ms
mininet> iperf
mininet> h1 ping -c10 h2
```

At startup, Mininet shows `(10.00Mbit 10ms delay)` on each link. `iperf` measured 9.50 Mbit/s, just below the 10 Mbit/s limit. The second value given by `iperf` (12.0 Mbit/s) is measured on the client side and also counts the data still waiting in the sending buffer, the real throughput is the one measured by the server. The average RTT of the pings is 40.7 ms, which matches the expected value: each link adds 10 ms, and the ICMP request and reply each cross two links ($2 \times 2 \times 10 = 40$ ms). The first ping is slightly longer (41.7 ms) because it goes through the controller.

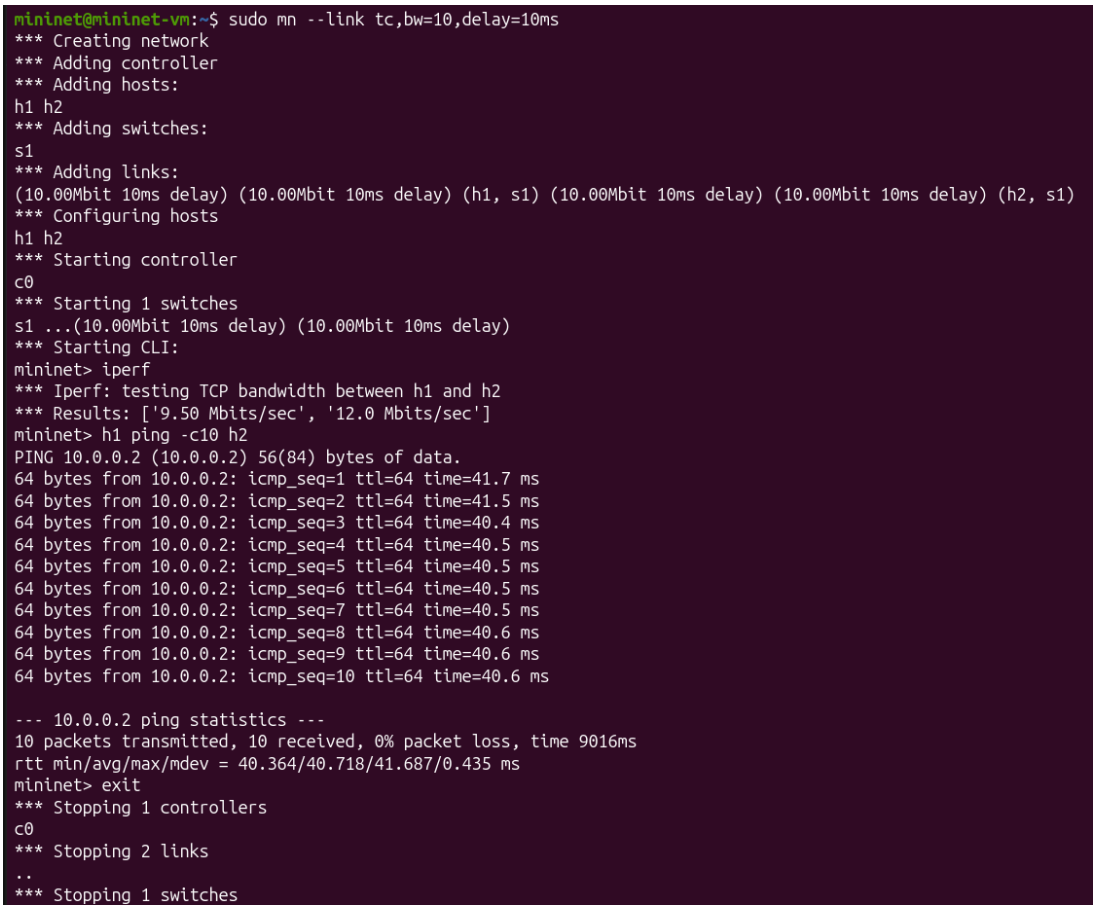


Figure 36: `-link tc,bw=10,delay=10ms`: `iperf` and `ping` results.

Topology of Figure 3 with the Python API. The following script creates the topology of Figure 3: host h1 is limited to 20% of the CPU with `CPULimitedHost`, and the link between h2 and s1 is limited to 10 Mbit/s with a 50 ms delay with `TCLink`.

At startup, Mininet shows that the limits are applied: `h1 (cfs 20000/100000us)` means that h1 can use 20 ms of CPU time every 100 ms, i.e. 20% of the CPU (h2 shows `-1`, i.e. no limit), and only the h2–s1 link shows `(10.00Mbit 50ms delay)`. `iperf` measured 9.23 Mbit/s, and the average RTT between h1 and h2 is 100.6 ms: the 50 ms delay is applied in each direction on the h2–s1 link, while the h1–s1 link has no delay ($2 \times 50 = 100$ ms).

```
mininet@mininet-vm:~/lab1$ cat topo_fig3.py
#!/usr/bin/env python3
# Part B.5 - Topology of Figure 3
# Run with: sudo python3 topo_fig3.py

from mininet.net import Mininet
from mininet.topo import Topo
from mininet.node import CPULimitedHost
from mininet.link import TCLink
from mininet.cli import CLI
from mininet.log import setLogLevel


class Fig3Topo(Topo):
    def build(self):
        s1 = self.addSwitch('s1')
        h1 = self.addHost('h1', ip='10.0.0.1/8', cpu=0.2)
        h2 = self.addHost('h2', ip='10.0.0.2/8')

        self.addLink(h1, s1)
        self.addLink(h2, s1, bw=10, delay='50ms')

topos = {'fig3': Fig3Topo}


if __name__ == '__main__':
    setLogLevel('info')
    net = Mininet(topo=Fig3Topo(), host=CPULimitedHost, link=TCLink)
    net.start()

    net.pingAll()
    CLI(net)
    net.stop()
```

Figure 37: Topology of Figure 3: Script `topo_fig3.py`.

```
mininet@mininet-vm:~/lab1$ sudo python3 topo_fig3.py
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (10.00Mbit 50ms delay) (10.00Mbit 50ms delay) (h2, s1)
*** Configuring hosts
h1 (cfs 20000/100000us) h2 (cfs -1/100000us)
*** Starting controller
c0
*** Starting 1 switches
s1 ...(10.00Mbit 50ms delay)
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
*** Starting CLI:
mininet> iperf h1 h2
*** Iperf: testing TCP bandwidth between h1 and h2
*** Results: ['9.23 Mbits/sec', '13.1 Mbits/sec']
mininet> h1 ping -c 5 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 100.414/100.607/101.160/0.281 ms
mininet>
```

Figure 38: Topology of Figure 3:`iperf` and `ping` results.

3.6 Part B.6: Adjustable Verbosity

Verbosity levels. The default verbosity level of Mininet is `info`. With `sudo mn -v debug`, Mininet prints every command it runs internally. This output shows, for instance, that the controller is started with `controller ptcp:6653`, that the veth pairs are created with `ip link add ... type veth peer name ...`, and that the switch is created with `ovs-vsctl`, with an active connection to the controller (`tcp:127.0.0.1:6653`) and a passive listening port `ptcp:6654`. With `sudo mn -v output`, only the CLI prompt is printed.

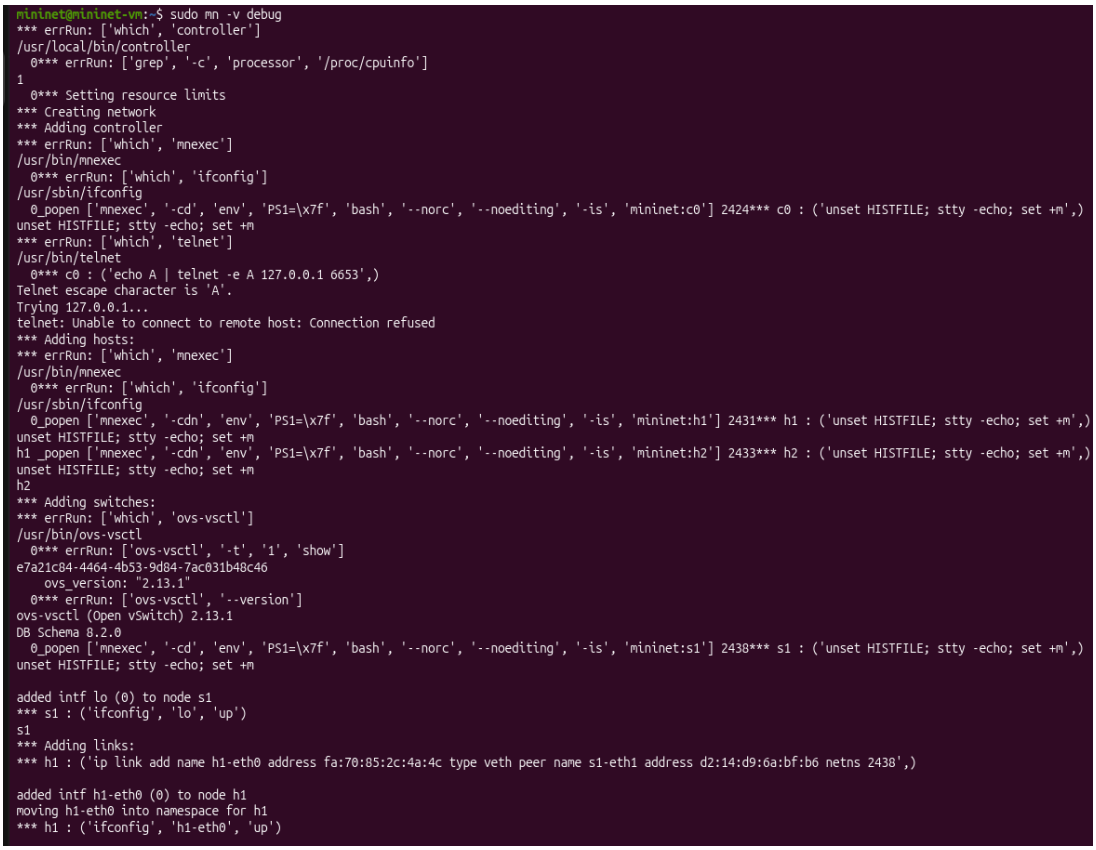


Figure 39: Extract of the output of `sudo mn -v debug`.



Figure 40: Extract of the output of `sudo mn -v debug`.

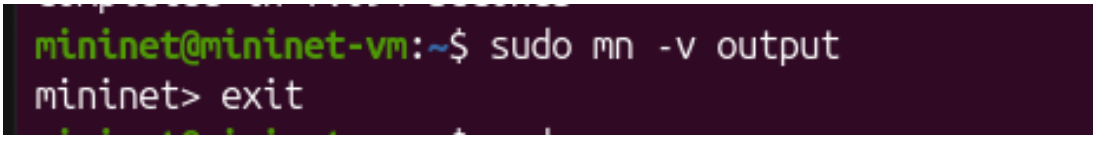


Figure 41: Output of `sudo mn -v output`.

xterms. The `-x` option opens an xterm for each host, switch and controller. Through the SSH session, `sudo mn -x` failed with `X11 connection rejected because of wrong authentication`, for the same reason as Wireshark in Part A. The xterms were opened with: sudo XAUTHORITY= HOME/.Xauthority mn -x



Figure 42: xterms opened by `mn -x` for h1, h2, s1 and c0.

Flow table from the switch xterm. In the xterm of s1, the command of the lab statement (`dpctl dump-flows tcp:127.0.0.1:6634`) failed with `Connection refused`: in this version of Mininet, the passive listening port of the first switch is 6654, as shown by the debug output. With port 6654, the switch answers with an empty flow table. After running `ping 10.0.0.2` in the xterm of h1, the same command shows several flow entries.



Figure 43: dpctl in the xterm of s1: port 6634 refused, empty table on port 6654.

The `dpctl dump-flows` command of the Mininet CLI gives the same result without needing an xterm or the IP address and port of the switch. Six rules were installed by the controller: two ICMP rules (Echo Request, type 8, and Echo Reply, type 0, already used 6 times) and four ARP rules (requests and replies in both directions), all with `idle_timeout=60`. Note that when the CLI command was first run more than one minute after the end of the ping, the table was empty: the rules had expired.

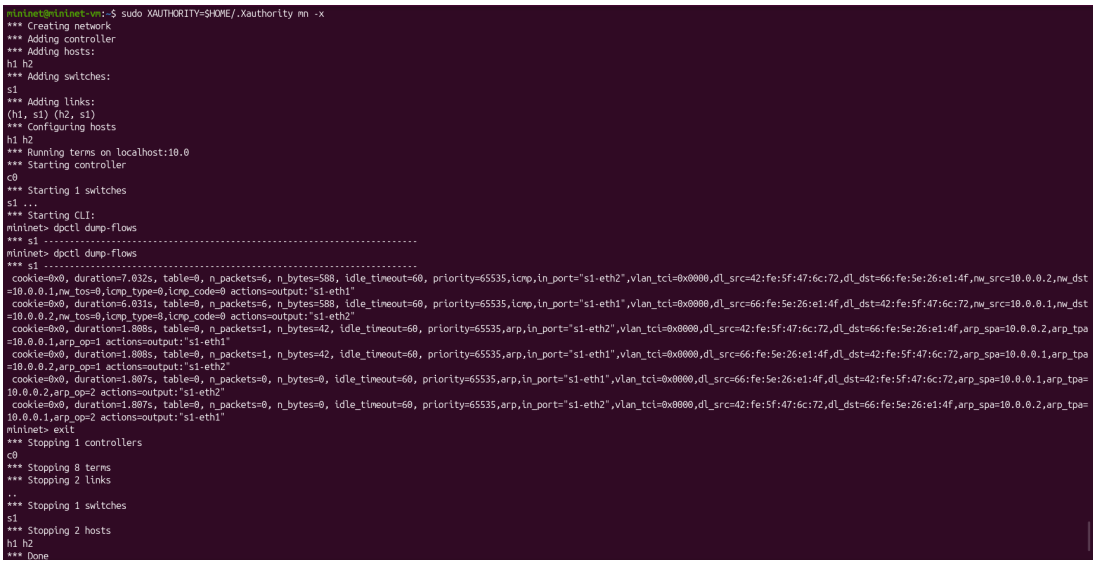


Figure 44: `dpctl dump-flows` from the Mininet CLI.

Root namespace. `ifconfig` in the xterm of h1 only shows `h1-eth0` and `lo`, while in the xterm of s1 it shows all the interfaces, including `eth0`: the switch xterm runs in the root namespace, as indicated by `(root)` in its title. The xterms closed automatically after `exit`.

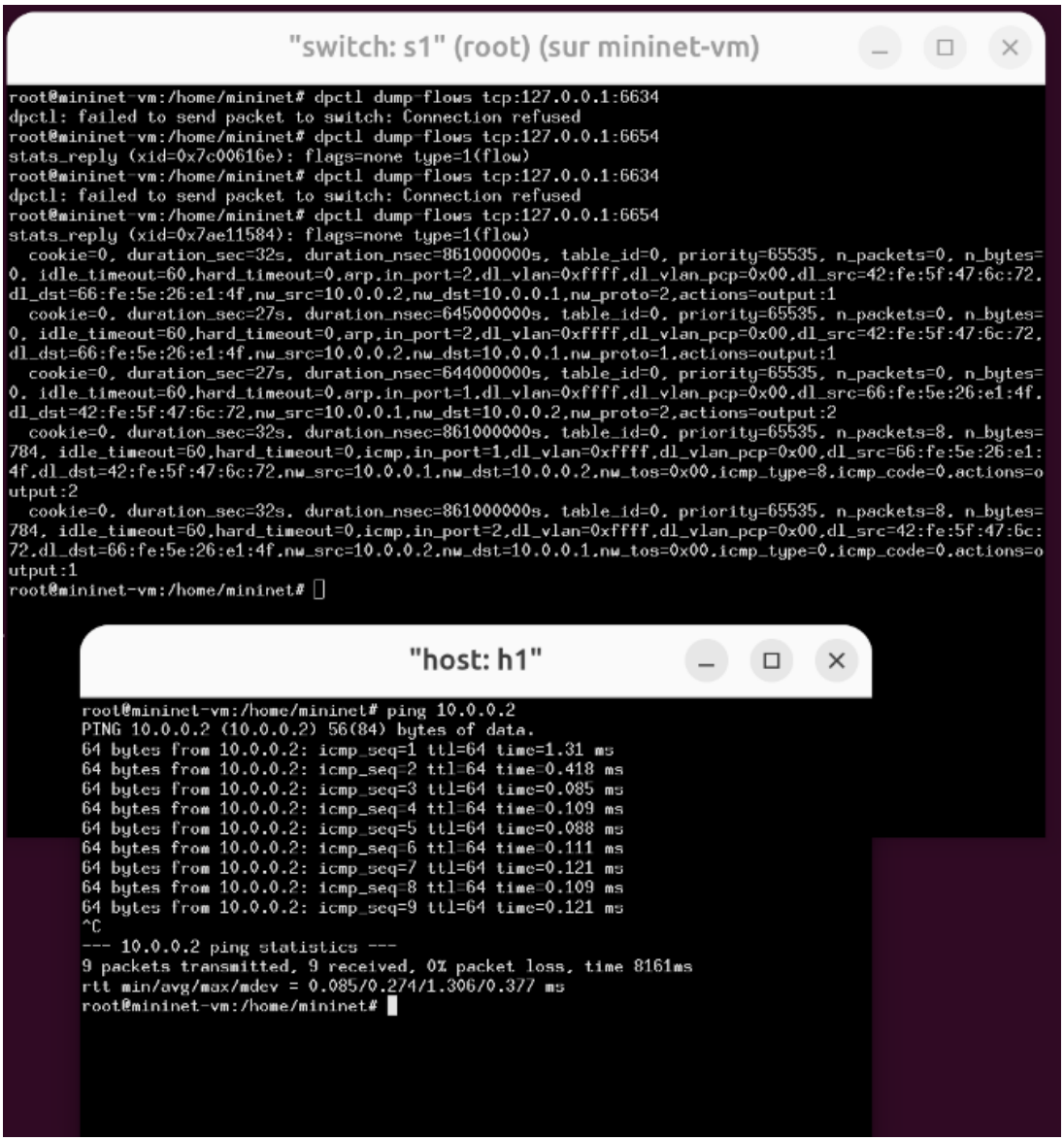


Figure 45: dpctl dump-flows in the xterm of s1 after a ping from h1: flow entries installed.

3.7 Part B.7: Python Interpreter

When a command of the Mininet CLI starts with `py`, it is executed by the Python interpreter. Each host, switch and controller is represented by a Python `Node` object that can be inspected and used directly.

The attribute `s1.listenPort` returns 6654, which confirms the passive listening port of the switch found in Part B.6.



Figure 46: `py` commands: expression, `locals()`, `dir(s1)`, `h1.IP()` and `h1.MAC()`.

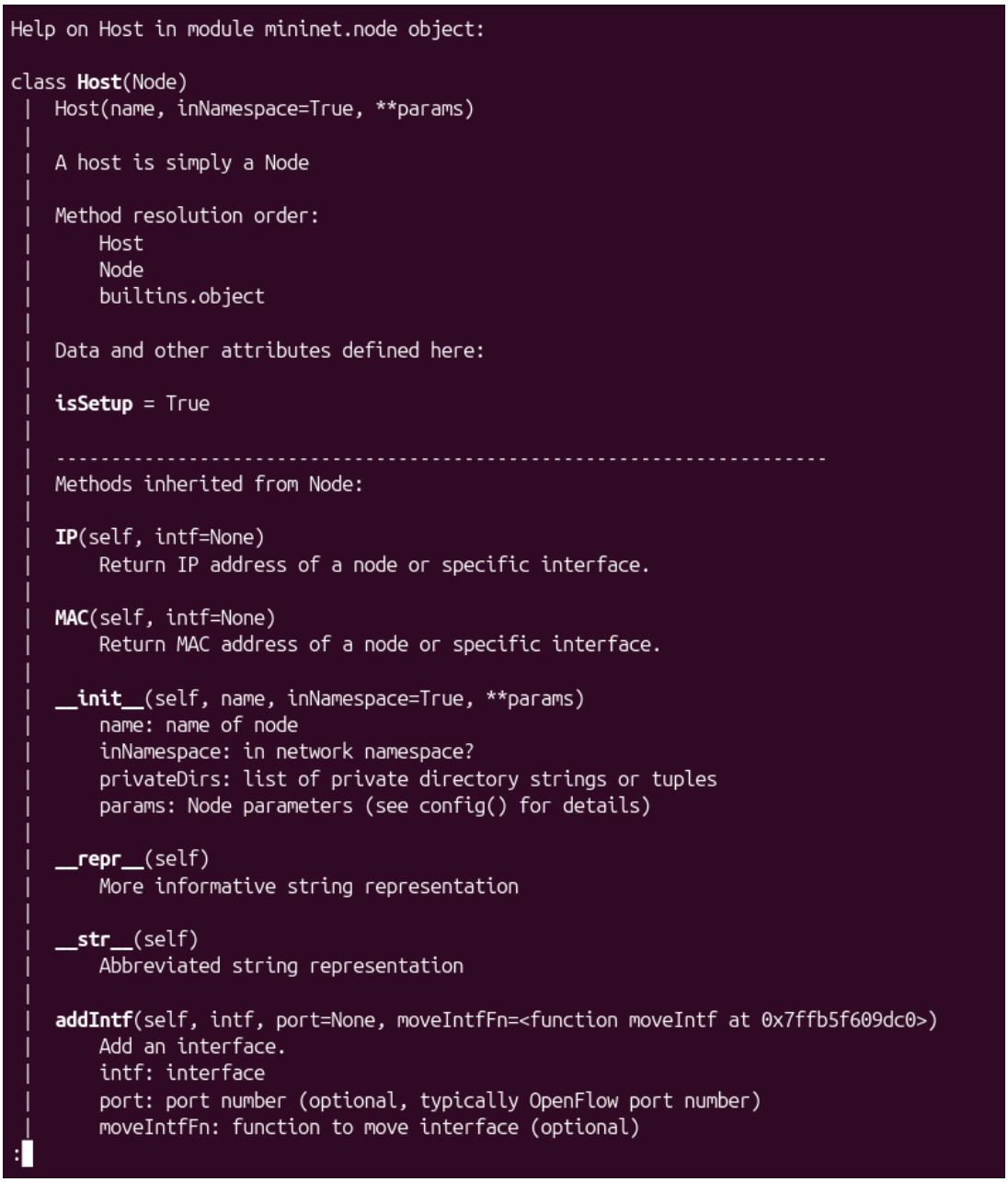


Figure 47: Documentation of the `Host` class displayed with `py help(h1)`.

3.8 Part B.8: Link Up/Down

The command `link s1 h1 down` disables both ends of the virtual Ethernet pair between s1 and h1, which is useful for fault tolerance tests. Wireshark was capturing with the display filter `openflow_1_0.type == 12`, which only shows the `OFPT_PORT_STATUS` messages (type 12 in OpenFlow 1.0).

Before the failure, h1 could ping h2. After `link s1 h1 down`, the ping failed immediately with `Network is unreachable`. After `link s1 h1 up`, the ping worked again (the first packet being slower, as it went through the controller again).

```
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ping -c 2 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=4.43 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.376 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.376/2.401/4.426/2.025 ms
mininet> link s1 h1 down
mininet> h1 ping -c 2 h2
ping: connect: Network is unreachable
mininet> link s1 h1 up
mininet> h1 ping -c 2 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=3.12 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.455 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.455/1.789/3.123/1.334 ms
mininet>
```

Figure 48: Pings before, during and after the link failure.

Three `OFPT_PORT_STATUS` messages were captured: one at the start of Mininet, one when the link was brought down and one when it was brought up again. These asynchronous messages are sent by the switch to inform the controller of a change on one of its ports.

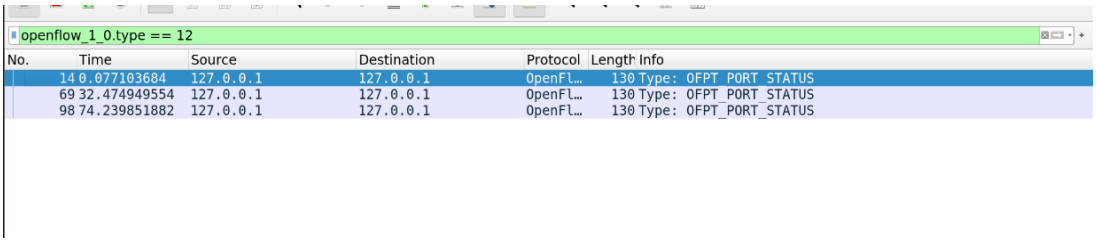


Figure 49: `OFPT_PORT_STATUS` messages sent by s1 to the controller.

Wireshark 4.4 does not decode the body of OpenFlow 1.0 `PORT_STATUS` messages (`Message`

`data not dissected yet`). The content was therefore read directly in the bytes of the message sent when the link was brought down, following the `ofp_port_status` structure of the OpenFlow 1.0 specification [3]: the port number is `0x0001` (`s1-eth1`, the port connected to h1), followed by the MAC address of the port and its name, and the *state* field is `0x00000001`, i.e. `OFPPS_LINK_DOWN`.

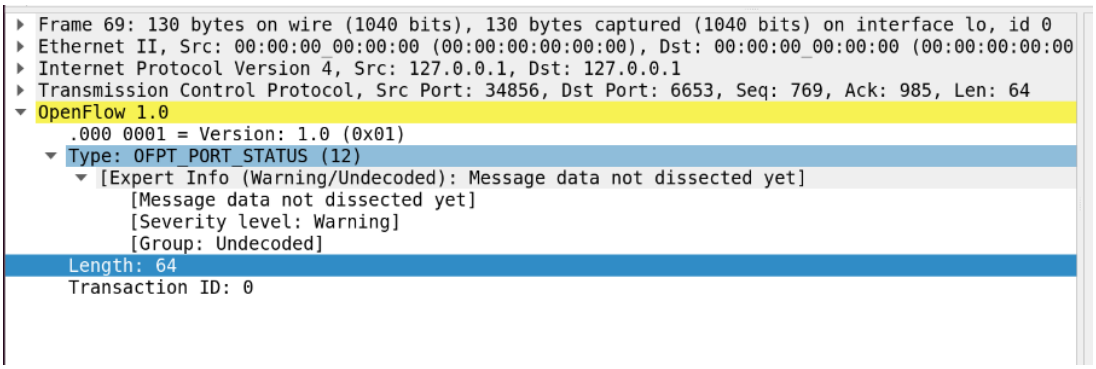


Figure 50: Bytes of the `PORT_STATUS` message sent when the link was brought down.

4 Conclusion

This lab observe concretely the separation between the data plane and the control plane in SDN: when a packet matches no rule, the switch sends it to the controller (`PACKET_IN`), which installs a rule (`FLOW_MOD`) so that the following packets are forwarded directly by the switch. We also used Mininet's built-in and custom topologies, link and CPU limits, the Python API and link failures.

References

[1] N. McKeown et al. OpenFlow: Enabling Innovation in Campus Networks. *ACM SIGCOMM CCR*, 38(2), 2008.

[2] B. Lantz, B. Heller, N. McKeown. A Network in a Laptop: Rapid Prototyping for Software-Defined Networks. *ACM HotNets*, 2010.

[3] Open Networking Foundation. OpenFlow Switch Specification, Version 1.0.0, 2009.

[4] Mininet Walkthrough. http://mininet.org/walkthrough/

[5] Mininet releases (VM images). https://github.com/mininet/mininet/releases

[6] Mininet Python API. http://mininet.org/api/annotated.html